

Faculté des Sciences Ben M'Sik — Université Hassan II de Casablanca

DÉPARTEMENT DE MATHÉMATIQUES ET INFORMATIQUE

Master DSBD & IA — Module : Algorithmes d'Optimisation

Subject:

## **Optimizer Comparison for Vision Transformers**

Fine-tuning DeiT-Tiny and ResNet-18 on CIFAR-10

Project by :

Hajar Alouani

Wahiba Bousyf

Hamza Chentouf

Année Universitaire : 2025-2026

## Table des Matières

|                                            |    |
|--------------------------------------------|----|
| 1. Introduction .....                      | 3  |
| 1.1 Project Objective .....                | 3  |
| 1.2 Models Overview .....                  | 3  |
| 1.2.1 DeiT-Tiny (Vision Transformer) ..... | 3  |
| 1.2.2 ResNet-18 (CNN Baseline) .....       | 3  |
| 1.3 Optimizers Overview .....              | 3  |
| 2. Literature Review .....                 | 5  |
| 3. Methodology .....                       | 6  |
| 3.1 Dataset: CIFAR-10 .....                | 6  |
| 3.2 Models and Pre-training .....          | 6  |
| 3.3 Experimental Protocol .....            | 6  |
| 3.4 Evaluation Metrics .....               | 7  |
| 4. Results .....                           | 8  |
| 4.1 DeiT-Tiny Results .....                | 8  |
| 4.2 ResNet-18 Results .....                | 9  |
| 5. Discussion .....                        | 10 |
| 5.1 Interpretation of Results .....        | 10 |
| 5.1.1 Best Validation Accuracy .....       | 10 |
| 5.1.2 Convergence Speed .....              | 10 |
| 5.1.3 Overfitting Gap .....                | 11 |
| 5.1.4 Training Time .....                  | 11 |
| 5.2 Interpretation of Results .....        | 12 |
| 5.3 Strengths and Weaknesses Summary ..... | 13 |
| 6. Conclusion .....                        | 14 |

# **1. Introduction**

## **1.1 Project Objective**

The primary objective of this project evaluates the performance of five optimization algorithms; AdamW, Lion, SGD with Momentum, AdaBelief, and RMSProp, on image classification tasks using a DeiT-Tiny Vision Transformer fine-tuned on the CIFAR-10 dataset and compares the results with a ResNet-18 CNN baseline under identical conditions. The main objective is to determine whether optimizer effectiveness depends on the model architecture and whether Vision Transformers and CNNs exhibit different optimization behaviors. The evaluation focuses on six criteria: convergence speed, gradient stability, generalization performance, learning-rate sensitivity, GPU memory usage, and training efficiency, with the goal of providing practical recommendations for optimizer selection in Vision Transformer applications.

## **1.2 Models Overview**

### **1.2.1 DeiT-Tiny (Vision Transformer)**

DeiT (Data-Efficient Image Transformer) is a lightweight Vision Transformer proposed by Touvron et al. (2021)[1] to improve data efficiency. Unlike the original ViT, which requires very large datasets, DeiT achieves strong performance on ImageNet using standard data augmentation and knowledge distillation. The DeiT-Tiny model has about 5.7 million parameters, with 12 attention heads, 3 transformer blocks, and  $16 \times 16$  patch embeddings for  $224 \times 224$  images. Unlike CNNs, Vision Transformers process images as sequences of patches and use self-attention to capture global relationships between image regions. This ability to model long-range dependencies improves representation learning but makes training more sensitive to hyperparameters such as the optimizer, weight decay, and learning rate schedule.

### **1.2.2 ResNet-18 (CNN Baseline)**

ResNet-18, introduced by He et al. (2015)[2], is a classical residual network with 18 layers and approximately 11.7 million parameters. It uses local  $3 \times 3$  convolutional filters organized in residual blocks with skip connections to mitigate the vanishing gradient problem. ResNet-18 represents the dominant paradigm in image classification prior to the Transformer era and serves as a well-calibrated reference point for evaluating how optimizer behavior differs between attention-based and convolution-based architectures.

## **1.3 Optimizers Overview**

Five optimization algorithms are evaluated in this study:

- **AdamW** (Loshchilov & Hutter, 2019)[3]: improves Adam by separating weight decay from gradient updates, leading to better regularization and more reliable adaptive learning rates. It is widely considered the standard optimizer for Transformer-based models.
- **Lion** (Chen et al., 2023)[4]: A memory-efficient optimizer discovered through evolutionary program search. Lion operates solely on the sign of the gradient update, reducing memory usage by eliminating the need to store second-moment estimates. It requires a learning rate approximately 10x smaller than AdamW.
- **SGD with Momentum** (Sutskever et al., 2013)[5]: The classical optimizer that applies a uniform learning rate to all parameters with Nesterov momentum. While historically dominant for CNN training, its lack of per-parameter adaptation makes it less suited for architectures with heterogeneous gradient scales such as Transformers.
- **AdaBelief** (Zhuang et al., 2020)[6]: An adaptive optimizer that scales the learning rate by the 'belief' in the gradient direction, defined as the inverse of the variance between the current gradient and its exponential moving average. AdaBelief achieves faster convergence and better generalization than Adam on several benchmarks.
- **RMSProp** (Hinton, 2012)[7]: A per-parameter adaptive optimizer that divides the learning rate by a running average of the squared gradient magnitudes. While effective for RNNs, RMSProp lacks momentum correction and decoupled weight decay, which can limit its stability on large-scale architectures.

## 2. Literature Review

Optimizer selection remains a critical challenge in deep learning, particularly with the introduction of Vision Transformers (ViTs), which exhibit greater sensitivity to training hyperparameters than CNNs.

**Adam**, introduced by Kingma and Ba (2017)[8], became a popular adaptive optimizer, but **AdamW** (Loshchilov & Hutter, 2019)[3] improved its performance by decoupling weight decay from gradient updates, making it especially effective for Transformer training. Studies on **ViT** and **DeiT** demonstrated that stable Transformer training relies heavily on adaptive optimizers such as AdamW, combined with carefully tuned weight decay and learning-rate schedules.

More recently, **Lion** (Chen et al., 2023)[4] challenged conventional optimizer design by using only the sign of gradient updates, reducing memory consumption while maintaining competitive performance, although it requires smaller learning rates. **AdaBelief** (Zhuang et al., 2020)[6] further refined adaptive optimization by estimating confidence in gradient directions, often achieving faster convergence and strong generalization. In contrast, **SGD with Momentum**, while highly successful for CNNs, generally underperforms on Transformer architectures because it lacks per-parameter adaptation. Previous research suggests that the heterogeneous gradient scales present in attention-based models make adaptive optimization particularly important, highlighting a potential interaction between optimizer choice and neural network architecture. This architectural dependency constitutes the central hypothesis investigated in the present study.

### 3. Methodology

#### 3.1 Dataset: CIFAR-10

CIFAR-10 (Krizhevsky, 2009)[9] consists of 60,000 color images of 32x32 pixels distributed across 10 mutually exclusive classes: airplane, automobile, bird, cat, deer, dog, frog, horse, ship, and truck. The dataset is split into 50,000 training images and 10,000 test images, with perfect class balance (6,000 images per class). CIFAR-10 is a standard benchmark for image classification research, offering sufficient complexity to distinguish optimizer behavior while remaining computationally tractable.

Since DeiT-Tiny expects 224x224 input images, all CIFAR-10 images were upsampled from 32x32 to 224x224 using bilinear interpolation via PyTorch's `torchvision.transforms.Resize`. Pixel values were normalized using the CIFAR-10 dataset mean (0.4914, 0.4822, 0.4465) and standard deviation (0.2470, 0.2435, 0.2616). Random horizontal flipping was applied during training as the sole data augmentation strategy.

#### 3.2 Models and Pre-training

Both models were initialized from ImageNet-1k pre-trained weights:

- **DeiT-Tiny**: loaded via the timm library (Wightman, 2019)[10] using the checkpoint 'deit\_tiny\_patch16\_224'. The classification head was replaced with a linear layer mapping from 192 dimensions to 10 classes.
- **ResNet-18**: loaded from PyTorch's torchvision model zoo (ResNet18\_Weights.IMAGENET1K\_V1). The fully connected layer was replaced with a linear layer mapping from 512 dimensions to 10 classes.

Fine-tuning all layers was performed rather than linear probing, as the target domain is sufficiently similar to ImageNet to benefit from gradient updates across all layers.

#### 3.3 Experimental Protocol

Each of the 10 experimental runs (5 optimizers x 2 models) was executed independently with fresh model weights loaded from the same pre-trained checkpoint. This ensures that each run is independent and that observed differences are attributable solely to the optimizer. The following fixed hyperparameters were shared across all runs:

- Number of epochs: 20
- Batch size: 256

- Loss function: Cross-Entropy with label smoothing epsilon = 0.1
- Gradient clipping: max norm = 1.0 (applied before optimizer step)
- Learning rate scheduler: Cosine Annealing with T\_max = 20 and eta\_min = 1e-6
- Hardware: NVIDIA Tesla T4 16GB (Kaggle platform)
- Framework: PyTorch 2.10.0, timm 1.0.x, CUDA 12.8

Table 1 summarizes the optimizer-specific hyperparameters used in each run:

| Optimizer           | Learning Rate | Momentum Params                 | Weight Decay | Scheduler |
|---------------------|---------------|---------------------------------|--------------|-----------|
| <b>AdamW</b>        | lr=1e-4       | $\beta_1=0.9$ , $\beta_2=0.999$ | wd=0.05      | Cosine    |
| <b>Lion</b>         | lr=1e-5       | $\beta_1=0.9$ , $\beta_2=0.99$  | wd=0.05      | Cosine    |
| <b>SGD+Momentum</b> | lr=1e-2       | mom=0.9, Nesterov               | wd=1e-4      | Cosine    |
| <b>AdaBelief</b>    | lr=1e-4       | $\beta_1=0.9$ , $\beta_2=0.999$ | wd=1e-4      | Cosine    |
| <b>RMSProp</b>      | lr=1e-4       | $\alpha=0.99$                   | wd=1e-4      | Cosine    |

Table 1: Optimizer-specific hyperparameters. Lion uses a 10x smaller LR as recommended by Chen et al. (2023).

### 3.4 Evaluation Metrics

The following metrics were recorded at each epoch for all 10 runs:

- Training loss and accuracy (CrossEntropy with label smoothing)
- Validation loss and accuracy (on held-out 10,000 test images)
- Validation F1-score (macro-averaged across 10 classes)
- L2 gradient norm (computed before gradient clipping)
- Epoch wall-clock time (seconds)

The summary statistics computed post-training include: best validation accuracy across all epochs, final (epoch 20) validation accuracy, best F1-score, total training time in minutes, and number of epochs required to first reach 80% validation accuracy.

## 4. Results

### 4.1 DeiT-Tiny Results

Table 2 presents the complete performance summary for DeiT-Tiny across all five optimizers after 20 epochs of fine-tuning on CIFAR-10.

| Optimizer           | Best Acc | Final Acc | Best F1 | Final Loss | Time (mn) | Ep→80% |
|---------------------|----------|-----------|---------|------------|-----------|--------|
| <b>AdamW</b>        | 0.9640   | 0.9610    | 0.9641  | 0.6250     | 23.8      | 1      |
| <b>Lion</b>         | 0.9637   | 0.9624    | 0.9637  | 0.6344     | 23.5      | 1      |
| <b>AdaBelief</b>    | 0.9568   | 0.9568    | 0.9568  | 0.6069     | 59.1      | 1      |
| <b>RMSProp</b>      | 0.1116   | 0.1000    | 0.0352  | 2.3026     | 47.2      | >20    |
| <b>SGD+Momentum</b> | 0.0768   | 0.0746    | 0.0775  | 2.4240     | 22.7      | >20    |

Table 2: DeiT-Tiny performance summary. Best results highlighted per column.

AdamW achieves the highest validation accuracy (96.40%) on DeiT-Tiny, followed closely by Lion (96.37%) and AdaBelief (95.68%). The performance gap between these adaptive optimizers and SGD+Momentum (7.68%) and RMSProp (11.16%) is dramatic, confirming that non-adaptive methods fail to optimize Vision Transformer architectures under standard fine-tuning conditions.

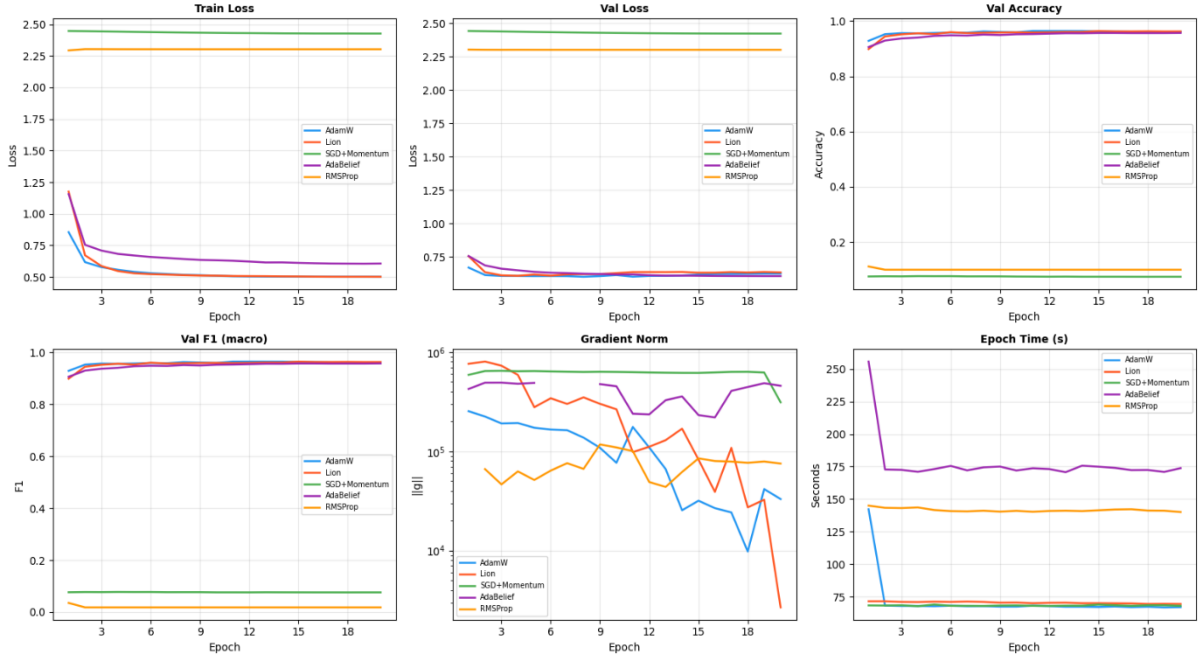


Figure 1: DeiT-Tiny training curves

Figure 1 shows the evolution of training loss, validation loss, validation accuracy, F1-score, gradient norm, and epoch time across 20 epochs for DeiT-Tiny. AdamW and Lion converge rapidly within the first 5 epochs to above 95% accuracy, while AdaBelief follows more slowly but ultimately reaches 95.68%. SGD+Momentum and RMSProp never converge, with loss curves remaining near the random-chance level (loss ~2.3–2.5, accuracy ~7–11%).

## 4.2 ResNet-18 Results

Table 3 presents the performance summary for ResNet-18. The pattern differs markedly from DeiT-Tiny, confirming the architecture-optimizer interaction hypothesis.

| Optimizer           | Best Acc | Final Acc | Best F1 | Final Loss | Time (mn) | Ep→80% |
|---------------------|----------|-----------|---------|------------|-----------|--------|
| <b>Lion</b>         | 0.9630   | 0.9630    | 0.9629  | 0.5984     | 47.2      | 1      |
| <b>AdamW</b>        | 0.9588   | 0.9573    | 0.9588  | 0.6231     | 48.1      | 1      |
| <b>AdaBelief</b>    | 0.8645   | 0.8636    | 0.8644  | 0.8783     | 23.3      | 7      |
| <b>RMSProp</b>      | 0.2780   | 0.2773    | 0.1448  | 2.0830     | 22.2      | >20    |
| <b>SGD+Momentum</b> | 0.0923   | 0.0923    | 0.0556  | 2.5044     | 24.1      | >20    |

Table 3: ResNet-18 performance summary.

On ResNet-18, Lion achieves the highest accuracy (96.30%), marginally outperforming AdamW (95.88%). Notably, AdaBelief reaches only 86.45% on ResNet-18 after 20 epochs, suggesting slower convergence on CNN architectures with the chosen hyperparameters. SGD+Momentum fails catastrophically on ResNet-18 as well, achieving only 9.23% accuracy, a result that departs from the historical literature where SGD dominates CNN benchmarks. RMSProp stabilizes around 27.80%, showing some learning but far below the adaptive optimizers.

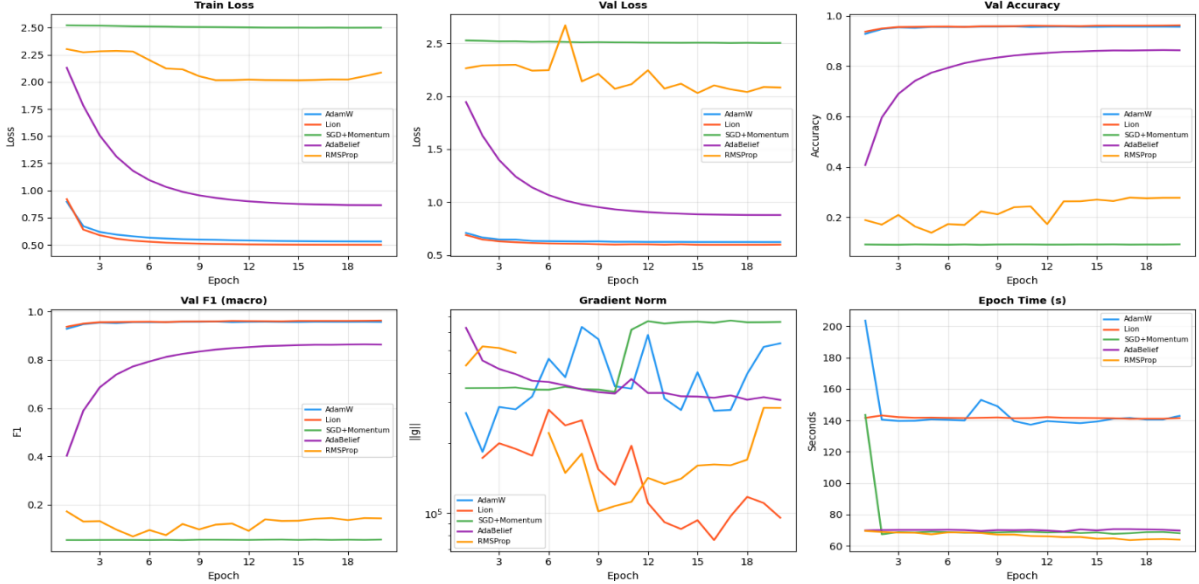


Figure 2: ResNet-18 training curves

ResNet-18 training results show clear differences between optimizers. **AdamW** and **Lion** perform best, converging quickly to low loss and achieving over **95% validation accuracy** within a few epochs. **AdaBelief** converges more slowly and reaches lower performance (~86% accuracy). **SGD with Momentum** fails to learn effectively, while **RMSProp** shows unstable training with loss spikes. Overall, AdamW and Lion provide the most stable and efficient optimization, while SGD and RMSProp underperform significantly in this setting.

## 5. Discussion

### 5.1 Interpretation of Results

#### 5.1.1 Best Validation Accuracy

Figure 3 shows a clear split in optimizer performance across both DeiT-Tiny and ResNet-18. **AdamW** and **Lion** consistently achieve the highest validation accuracy (around 96% for DeiT-Tiny and ~96% for ResNet-18), forming a strong top-performing group. **AdaBelief** performs moderately well on DeiT-Tiny but drops notably on ResNet-18, indicating sensitivity to model architecture.

In contrast, **SGD with Momentum** and **RMSProp** perform poorly on both models, staying close to random-chance accuracy. Overall, there is a large performance gap between the best and worst optimizers, highlighting how critical optimizer choice is for successful training in both Vision Transformers and CNNs.

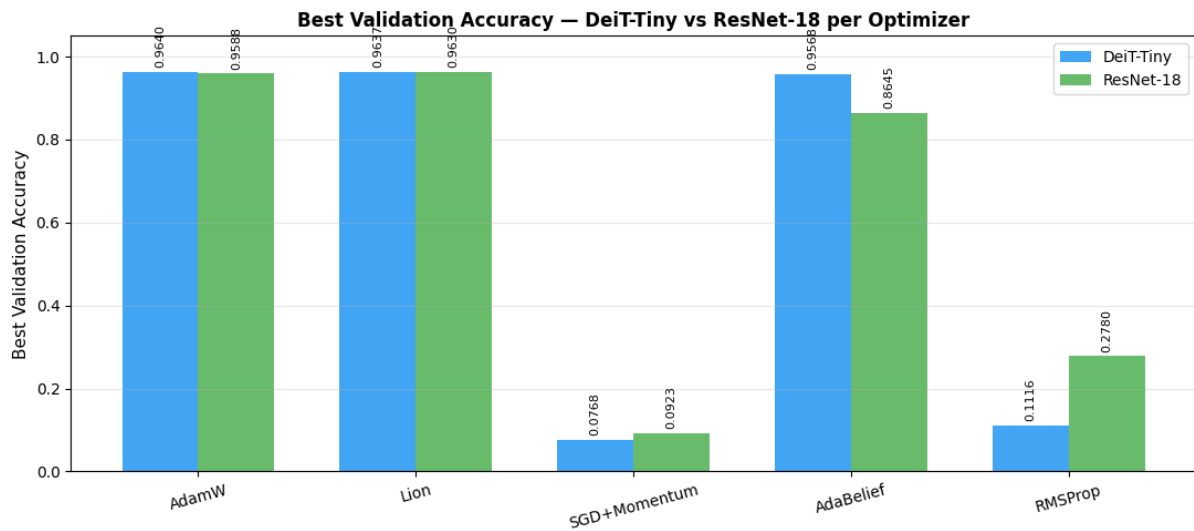


Figure 3: Best Validation Accuracy

#### 5.1.2 Convergence Speed

Figure 4 compares how quickly optimizers reach different accuracy levels on DeiT-Tiny and ResNet-18. AdamW and Lion converge very quickly on both models, reaching high accuracy thresholds within the first few epochs, with AdamW slightly faster at the highest levels. AdaBelief converges more slowly and shows a clear drop in efficiency on ResNet-18, where it struggles to reach higher accuracy thresholds within the training budget.

In contrast, SGD with Momentum and RMSProp fail to reach any meaningful accuracy thresholds. Overall, the results show that convergence speed depends not only on the optimizer itself but also on how it interacts with the underlying model architecture.

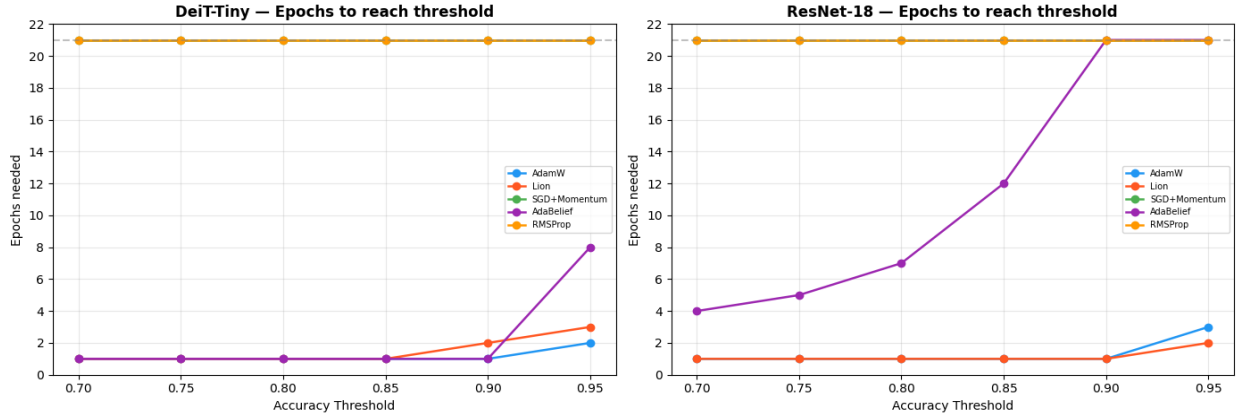


Figure 4: Convergence Speed

### 5.1.3 Overfitting Gap

Figure 5 analyzes generalization by comparing training and validation accuracy gaps across optimizers. On **DeiT-Tiny**, AdamW shows mild overfitting over time, with a small but growing positive gap as training accuracy approaches near-perfect levels. In contrast, **Lion** and **AdaBelief** maintain almost no overfitting, suggesting stronger implicit regularization effects, especially for Lion.

On **ResNet-18**, **AdaBelief** behaves unusually by often showing higher validation than training accuracy, likely due to slower convergence rather than true generalization gains. **RMSProp** is unstable, with large fluctuations in the accuracy gap, indicating inconsistent learning behavior. Overall, the results suggest that some optimizers (especially Lion and AdaBelief) may generalize more smoothly, while AdamW tends to slightly overfit and RMSProp remains unreliable.

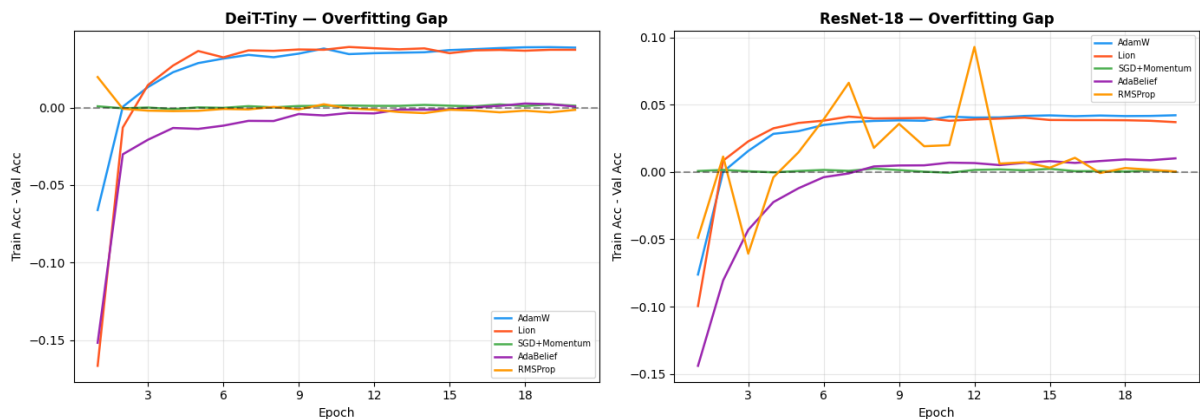


Figure 5: Overfitting Gap

### 5.1.4 Training Time

Figure 6 compares training time across optimizers and architectures. On **DeiT-Tiny**, most optimizers run at similar speed (~22–24 minutes total), except **AdaBelief**, which is significantly slower due to its additional computations and instability handling.

On **ResNet-18**, **AdamW** and **Lion** become the slowest (~47–48 minutes), while the other optimizers are faster but deliver much worse accuracy. This shows that faster training does not necessarily mean better performance. Overall, AdamW and Lion are more computationally expensive on larger models but provide the best trade-off between speed and accuracy.

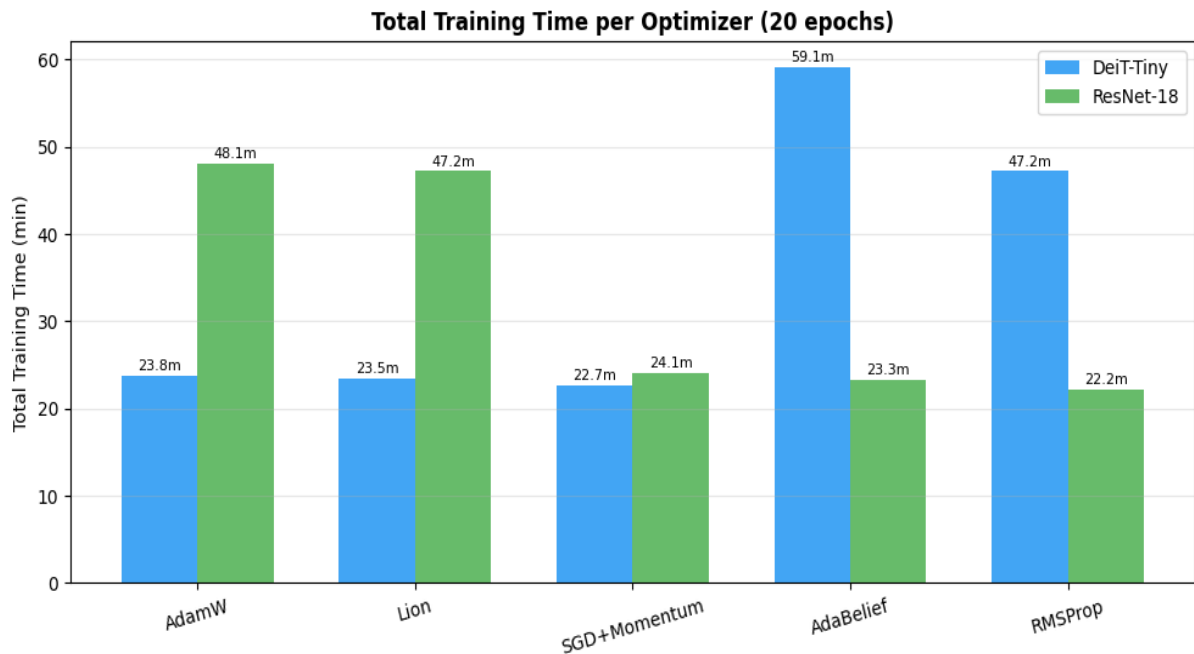


Figure 6: Training Time

## 5.2 Interpretation of Results

The results show that AdamW and Lion consistently achieve the best performance across both DeiT-Tiny and ResNet-18, confirming their suitability for modern deep learning models. AdamW provides the most stable and reliable convergence with strong generalization, while Lion matches or slightly exceeds its performance while offering lower memory usage and implicit regularization, though it requires careful learning-rate tuning.

AdaBelief performs well on DeiT-Tiny but degrades significantly on ResNet-18, indicating strong sensitivity to model architecture and hyperparameters. SGD with Momentum fails to converge in both settings due to its lack of adaptive updates, which makes it unsuitable for fine-tuning pre-trained models. Similarly, RMSProp shows unstable training behavior and poor convergence, limiting its effectiveness for these architectures.

Overall, the study confirms that adaptive optimizers, particularly AdamW and Lion, are far more effective for both Vision Transformers and CNNs, and that optimizer performance strongly depends on the interaction between the optimization method and the model architecture.

### **5.3 Strengths and Weaknesses Summary**

AdamW achieves the best overall performance on DeiT-Tiny (96.40%) with stable and consistent convergence across both architectures. It exhibits only mild overfitting after epoch 15 and remains the most reliable default choice for Vision Transformer fine-tuning. Its wide framework support and proven stability make it the strongest general-purpose optimizer in this study.

Lion delivers the highest performance on ResNet-18 (96.30%) and remains nearly equivalent to AdamW on DeiT-Tiny (96.37%). It offers a significant advantage in memory efficiency (approximately 33% reduction in optimizer state storage) and maintains a near-zero overfitting gap, indicating strong implicit regularization. However, it is highly sensitive to learning rate selection and requires values roughly an order of magnitude smaller than AdamW for stable training, making it particularly suitable in memory-constrained environments.

AdaBelief shows competitive performance on DeiT-Tiny (95.68%) but a substantial drop on ResNet-18 (86.45%), indicating clear architecture sensitivity. While it demonstrates controlled generalization behavior and a near-zero overfitting gap, it suffers from slower convergence and higher computational cost. Occasional gradient instability also necessitates careful use of gradient clipping. It may become more competitive under extended training schedules.

SGD with Momentum performs poorly in both settings, failing to converge effectively in the fine-tuning scenario (7–9% accuracy). Its lack of adaptive learning rates makes it unsuitable for transfer learning without additional techniques such as layer-wise learning rate tuning and warm-up strategies. As a result, it is not recommended as a direct replacement for adaptive optimizers in Vision Transformer training.

RMSProp exhibits unstable and unreliable behavior across both architectures, including erratic loss trajectories and gradient explosions. These instabilities prevent meaningful convergence under standard configurations. Significant hyperparameter redesign would be required for any competitive performance, making it unsuitable for practical ViT or CNN fine-tuning in its default form.

## 6. Conclusion

This study presented a systematic comparison of five optimization algorithms for fine-tuning a DeiT-Tiny Vision Transformer and a ResNet-18 convolutional neural network on the CIFAR-10 dataset. The results demonstrate that optimizer selection plays a critical role in determining convergence speed, training stability, and final model performance.

The experiments show that adaptive optimization methods are essential for successful fine-tuning of modern deep learning architectures. AdamW and Lion consistently achieved the best results across both models, whereas SGD with Momentum and RMSProp failed to converge effectively under the chosen transfer learning configuration. These findings indicate that non-adaptive optimization strategies are generally unsuitable for fine-tuning pre-trained models without substantial modifications to the learning-rate schedule and optimization procedure.

Among all evaluated optimizers, **AdamW** emerged as the most reliable choice, achieving the highest validation accuracy on DeiT-Tiny (96.40%) while maintaining excellent performance on ResNet-18 (95.88%). Its stable convergence behavior and well-established effectiveness confirm its position as the preferred default optimizer for Vision Transformer fine-tuning. **Lion** delivered nearly identical accuracy on both architectures (96.37% on DeiT-Tiny and 96.30% on ResNet-18) while requiring significantly less optimizer-state memory. This makes Lion an attractive alternative in memory-constrained environments.

**AdaBelief** produced competitive results on DeiT-Tiny but exhibited slower convergence and lower performance on ResNet-18. Although it incurred the highest computational cost, its strong generalization characteristics suggest that it may become more competitive under longer training schedules. In contrast, **SGD with Momentum** and **RMSProp** demonstrated poor convergence and unstable optimization dynamics, highlighting their limitations in transfer learning scenarios involving pre-trained Vision Transformers and CNNs.

Overall, the results confirm that the interaction between optimizer and architecture is an important factor in deep learning performance. While adaptive optimizers consistently outperform non-adaptive methods, their relative effectiveness varies across architectures. Based on the experimental evidence, AdamW remains the safest and most effective default choice for Vision Transformer fine-tuning, whereas Lion offers a compelling balance between accuracy and memory efficiency. These findings provide practical guidance for researchers and practitioners seeking to optimize the performance of Transformer-based image classification models under different computational constraints.

- [1] H. Touvron, M. Cord, M. Douze, F. Massa, A. Sablayrolles, and H. Jégou, “Training data-efficient image transformers & distillation through attention,” Jan. 15, 2021, *arXiv*: arXiv:2012.12877. doi: 10.48550/arXiv.2012.12877.
- [2] K. He, X. Zhang, S. Ren, and J. Sun, “Deep Residual Learning for Image Recognition,” Dec. 10, 2015, *arXiv*: arXiv:1512.03385. doi: 10.48550/arXiv.1512.03385.
- [3] I. Loshchilov and F. Hutter, “Decoupled Weight Decay Regularization,” Jan. 04, 2019, *arXiv*: arXiv:1711.05101. doi: 10.48550/arXiv.1711.05101.
- [4] X. Chen *et al.*, “Symbolic Discovery of Optimization Algorithms,” May 08, 2023, *arXiv*: arXiv:2302.06675. doi: 10.48550/arXiv.2302.06675.
- [5] I. Sutskever, “On the importance of initialization and momentum in deep learning”.
- [6] J. Zhuang *et al.*, “AdaBelief Optimizer: Adapting Stepsizes by the Belief in Observed Gradients,” Dec. 20, 2020, *arXiv*: arXiv:2010.07468. doi: 10.48550/arXiv.2010.07468.
- [7] G. Hinton, “Neural Networks for Machine Learning”.
- [8] D. P. Kingma and J. Ba, “Adam: A Method for Stochastic Optimization,” Jan. 30, 2017, *arXiv*: arXiv:1412.6980. doi: 10.48550/arXiv.1412.6980.
- [9] A. Krizhevsky, “Learning Multiple Layers of Features from Tiny Images”.
- [10] R. Wightman, *PyTorch Image Models*. (Jun. 20, 2026). Python. doi: 10.5281/zenodo.4414861.